SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Techniques for Using Neural Architecture Search in Federated Learning

Max Coetzee

SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY — INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

Techniques for Using Neural Architecture Search in Federated Learning

Techniken um Neurale Architektursuche für Föderatives Lernen zu nutzen

Author:	Max Coetzee
Examiner:	Ali Sunyaev
Supervisor:	Nick Henze, Niclas Kannengießer
Submission Date:	30.07.2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 30.07.2026 Max Coetzee

Abstract

Neural architecture search (NAS) automates the design of neural network architectures, while federated learning (FL) enables clients to train models collaboratively without centralizing their data. Combining them can tailor architectures to federated tasks and devices, but distributing architecture search introduces constraints that centralized NAS does not face. Existing research proposes many solutions, yet their mechanisms are commonly embedded within complete NAS-in-FL methods. This makes it difficult to determine which mechanism addresses which challenge and under which conditions it can be reused.

To consolidate this knowledge, this thesis examined how solutions address recurring challenges in the NAS-in-FL literature. We conducted a concept-centric literature review and thematic analysis of 48 publications identified through a Scopus search and backward and forward citation searching. For each publication, we extracted NAS design choices, FL system characteristics, challenges, and independently deployable solution mechanisms. We then mapped the mechanisms to the challenges they address and assessed their conceptual applicability by comparing their prerequisites with the properties of target NAS-in-FL scenarios.

The analysis identified ten challenges across five themes: client resource constraints, client heterogeneity, decentralized search state, participation dynamics, and search security. Across these challenges, we synthesized 34 solution mechanisms. The findings show that challenge relevance depends on which search functions rely on clients and that solution mechanisms can address several challenges, introduce further prerequisites or costs, and interact with other mechanisms. Their reuse is therefore context-dependent and requires matching the mechanism to the NAS design, FL system characteristics, and other mechanisms in the configuration. Separating mechanisms from complete methods provides a structured basis for comparing and designing NAS-in-FL methods. Because the reviewed evaluations cover heterogeneous and often incompletely described configurations, the synthesis supports conceptual comparison rather than a ranking of effectiveness. Controlled empirical evaluations under explicit operating conditions remain necessary.

Contents

1 Introduction

Deep learning has seen widespread adoption over the last decade and is being applied to an increasingly diverse set of tasks. Applying deep learning traditionally involves a team of deep learning and domain experts who tailor a neural network architecture to a specific task through a lengthy process of trial and error. To automate this process, researchers invented *Neural Architecture Search* (NAS) [10] methods. NAS methods employ diverse strategies to automatically search for a neural network architecture for a given deep learning task within an architecture search space.

Independently, but in parallel to NAS, researchers developed a machine learning approach called *Federated Learning* (FL) [34], which enables training an ML model on data distributed across a set of clients without sharing their data. The model is distributed from the server to the clients, which train the model on their local data. The clients then send the model weight updates to the server, which aggregates them and redistributes them to the clients. This process is repeated for several rounds. FL enhances the privacy of client data and enables training even when clients' data is too large to transfer effectively to a central training site or cannot be shared due to regulatory or privacy constraints.

There has been significant interest in using NAS in FL over the last couple of years, spawning *NAS-in-FL methods* [16], but using NAS methods in FL is not straightforward. Most NAS methods were developed for a centralised setting and can not be applied directly in FL. For example, in a centralised setting, NAS methods typically run on worker nodes connected via low-latency, high-bandwidth links. However, in FL, a NAS method may run on clients connected via an unreliable, high-latency, low-bandwidth network. This creates a *challenge* when transferring the weights of large candidate models between clients and the server to coordinate an architecture search. Specialized solutions are needed to overcome such challenges for NAS-in-FL methods, such as only transferring encodings of candidate architectures to clients [44].

The NAS-in-FL literature proposes many solutions to address these challenges, but their mechanisms are often obscured because they are embedded in a NAS-in-FL method. Earlier reviews classify NAS-in-FL methods by properties such as when search occurs and which objectives it optimizes [76], summarize selected methods within broader reviews [23], or discuss NAS as part of multi-objective FL [14]. More recent work supports a more fine-grained comparison through a generic NAS-in-FL pipeline, design elements, and method archetypes [16]. These contributions organize whole NAS-in-FL methods and their configurations, but solution and the challenges they address remain scattered across the literature.

Developing NAS-in-FL methods that work as intended requires a coherent understanding of the challenges introduced by federated search and the available solutions for addressing them. Without this understanding, researchers cannot readily compare proposed solutions, determine which challenges a solution leaves unresolved, or build systematically on earlier work. Practitioners may consequently select a method whose search procedure is infeasible or whose architecture selection becomes unreliable under the conditions of their FL system. The fragmented knowledge about solutions and the challenges they address limits both cumulative research and informed use of NAS in FL. A consolidated account is needed to support the assessment and development of NAS-in-FL methods. We therefore ask:

How do solutions address recurring challenges in the NAS-in-FL literature?

To answer this question, we conduct a concept-centric literature review and thematic analysis of 48 publications. We identify relevant work through a Scopus [9] search and backward and forward citation searching, then inductively extract relevant NAS method types and FL system characteristics, recurring challenges, and independently reusable solution mechanisms. We map each mechanism to the challenges it addresses and assess its conceptual applicability by comparing its prerequisites with the NAS design choices and FL system characteristics of a target configuration.

The analysis identifies ten challenges across five themes: client resource constraints, client heterogeneity, decentralized search state, participation dynamics, and search security. Across these challenges, we synthesize 34 solution mechanisms. A mechanism may address multiple challenges, interact with other mechanisms, and introduce additional prerequisites or trade-offs.

Our central finding is that solution mechanisms are reusable only when the targeted FL system characteristics satisfy their prerequisites. Separating these mechanisms from the complete NAS-in-FL methods in which they appear provides a structured way to characterize targeted FL system characteristics, identify the challenges created by its distributed dependencies, and select compatible mechanisms. Because the published evaluations cover heterogeneous and often incompletely described configurations, this synthesis supports conceptual comparison rather than a ranking of effectiveness; controlled empirical evaluations under explicit operating conditions remain necessary.

This thesis is structured as follows. Chapter 2 introduces NAS, FL, and their combination in NAS-in-FL. Chapter 3 describes the literature selection, thematic analysis, relationship mapping, and applicability assessment. Chapter 4 reviews prior NAS-in-FL syntheses and positions this review. Chapter 5 presents the ten identified challenges and 34 solution mechanisms across five themes. Chapter 6 interprets the findings and discusses their implications, limitations, and directions for future research. Finally, Chapter 7 summarizes the thesis and its central conclusion.

2 Background

2.1 Neural Architecture Search

Neural architecture search (NAS) automates the design of neural network architectures. A NAS method combines a search space, a search strategy, and a performance-estimation strategy [10]. The search space defines the candidate architectures. The search strategy selects candidates, and the performance-estimation strategy supplies the evidence used to compare them.

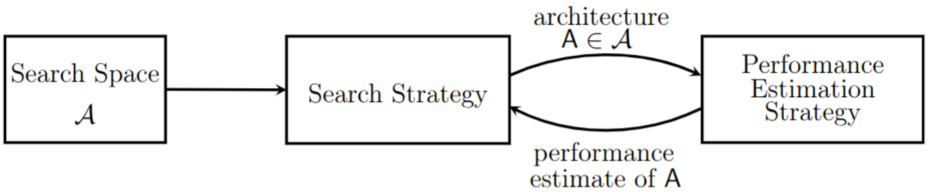

Figure 2.1: Overview of the three components of NAS from [10].

2.1.1 NAS Method Type

A NAS method type consists of a search-strategy type, search-space type, and performance-estimation type [10, 53] and helps us qualify for what kind of NAS methods challenges are applicable. This should not be confused with a concrete NAS method.

Search strategies in this review are primarily categorized as black-box optimization or supernet-based methods. Black-box optimization strategies include reinforcement learning, evolutionary algorithms (including genetic algorithms), and Bayesian optimization. Black-box optimization strategies evaluate distinct candidate architectures.

Supernet-based methods instead train candidates within a shared network. They are differentiable when gradients optimize continuous architecture parameters and non-differentiable when the search selects discrete subnetworks without continuous architecture parameters.

Search spaces are categorized as cell-based, hierarchical, chain-structured, or other. Cell-based spaces search reusable computational blocks, hierarchical spaces compose structures across multiple levels, and chain-structured spaces vary a sequence of layers. A search space also has capabilities that a solution may require, such as being supernet-encodable, elastic, progressively prunable, modular, discretely encodable, or divisible into trainable segments. These capabilities are separate from search-space type because spaces of the same type may support different operations.

Performance estimation is categorized as observed performance after independent candidate training, one-shot estimation with inherited shared weights, estimation with weights inherited from another candidate, surrogate-predicted performance, or another estimator [10, 53]. Independent training may use a full or reduced budget. Multi-fidelity evaluation compares estimates obtained at several budgets and may use observed or predicted performance. It therefore describes how evaluation resources are allocated rather than a separate source of performance evidence.

2.2 Federated Learning

Federated learning (FL) enables clients to train a model collaboratively while keeping their raw data local. A server distributes a global model to selected clients, which train it locally and return updates for aggregation [34].

While the FL literature distinguishe between broad categories, we require more fine-grained FL system characteristics that can vary independently to qualify the applicability of challenges [21, 5]:

Data heterogeneity distinguishes homogeneous client distributions from heterogeneous distributions, including differences in labels, features, domains, tasks, or time.

Data balance distinguishes balanced from imbalanced quantities of client data.

Client computation resources record whether compute and memory capacities are homogeneous or heterogeneous. Resource homogeneity does not imply sufficient capacity, so absolute limits form an additional condition.

Network environment records whether client bandwidth and latency are homogeneous or heterogeneous and whether they impose a binding communication constraint.

Federation environment distinguishes stable participation from dynamic participation caused by selection, changing availability, dropout, or delayed updates.

Deployment environment distinguishes a uniform deployment requirement from heterogeneous client- or group-specific requirements.

Trust and privacy requirements distinguish reliance on data locality from additional confidentiality or integrity requirements.

2.3 NAS in FL

In comparison to centralized NAS, the architecture search process is performed in a federated manner in NAS in FL. This usually means that candidate architectures get evaluated on clients. Candidate generation, client assignment, candidate evaluation, search-state updates, weight sharing, and architecture selection may be divided between the server and clients in different ways [16].

Clients can consequently exchange model weights, architecture variables, candidate encodings, or performance evidence. The chosen NAS method determines which search operations and state are distributed, while the FL system determines the data, resources, participation, deployment requirements, and trust conditions under which they operate.

3 Method

We conducted a concept-centric literature review following [50] and used thematic analysis following [2] to examine how NAS design choices and FL system characteristics give rise to challenges and how solution address those challenges under stated conditions.

3.1 Literature Selection

We start our literature selection with a search across titles, abstracts and keywords of English-language publications indexed by Scopus using the query `"federated learning" AND "neural architecture search"` [9]. A publication is eligible when it describes a method in which NAS is performed in FL. We excluded publications that address only NAS or FL independently and publications in which centrally completed NAS merely preceeds FL without NAS. Literature reviews are excluded from the analysis and discussed in related work.

For inclusion in our thematic analysis, a publication must provide evidence for at least one challenge or solution. A challenge meets this threshold when the publication connects a condition of FL to a consequence for architecture search. A solution meets the threshold when the described procedure contains a deliberate intervention whose effect on the NAS-in-FL process can be identified. General claims that an approach addresses concerns such as heterogeneity, efficiency or privacy without a suffienctly detailed explanation do not meet this threshold.

We extend this initial set of literature through backward and forward citation searching of the eligible publications. Newly identified publications are screened using the same criteria, and citation searching ends when an iteration adds no additional eligible publications.

Table 3.1: Literature selection process and the number of publications retained after each decision. The final set contains the publications used as evidence for the challenges and solutions in Chapter 5.

Step	**Selection decision**	**Publications**
1	Scopus query	206
2	NAS performed in FL	66
3	Reviews excluded	63
4	Citation searching	69
5	Duplicates removed	66
6	Challenge or solution evidence	48

3.2 Literature Analysis

We analyzed the literature in two stages. First, we extracted NAS method types and FL system characteristics and developed challenge and solution themes. Second, we mapped the relationships among these properties, challenges, and solutions and assessed the conditions that bound the applicability of each solution.

3.2.1 Extracting NAS Method Types, FL System Characteristics, Challenges, and Solutions

We read every eligible publication for familiarization, coded relevant passages inductively following [2], and used a concept matrix for initial coding [50]. For each extract, we recorded the source and page as well as the NAS method type and FL system characteristics. A challenge code expressed a relationship between a NAS design choice or FL system characteristic and its consequence for NAS in FL. A solution code recorded an intervention, how it changed the NAS-in-FL process, and its intended effect on a challenge. We also recorded prerequisites and trade-offs when the publication reported them.

We grouped challenge codes when they described the same consequence for architecture search and grouped solution codes when their mechanisms were conceptually similar. Similar terminology alone did not justify grouping. We retained a challenge when it made one coherent analytical claim and a solution when it contained one independently deployable mechanism. General NAS or FL concerns without a distinct consequence for NAS-in-FL were discarded.

After familiarization and initial coding, we iterated over stages three to five of thematic analysis to develop and refine the challenge and solution themes, as summarized in Table 3.2 [2]. While reviewing themes for internal homogeneity and external heterogeneity, we returned to the coded extracts and the corpus as needed. We merged contextual duplicates, split independent claims or mechanisms, and clarified boundaries between related themes. Refinement ended when another review identified no justified structural merger, split, or discard.

Table 3.2: Thematic analysis stages used to iteratively conceptualize challenges and solutions from the codes.

Stage	Main activity	Output
3. Generating themes	Group challenge codes that describe related consequences and solution codes that describe related mechanisms.	Candidate challenge and solution concepts with their supporting codes.
4. Reviewing themes	Compare each candidate concept with its codes and extracts, then assess its coherence and distinction from other concepts against the complete corpus.	Revised concepts and code assignments.
5. Defining and naming themes	Define each challenge through its activating condition and consequence and each solution through its intervention and intended effect.	Final concepts with clear names, definitions, and boundaries.

3.2.2 Mapping Relationships and Assessing Applicability

We linked a solution to a challenge only when a publication presented the solution mechanism as a response to that challenge. The presence of both in the same publication did not establish a relationship.

Next, we examined whether each solution mechanism could be applied under other combinations of NAS design choices and FL system characteristics. We identified the operations, representations, and system properties required by the mechanism and compared them with those available in the target configuration. We considered the mechanism conceptually compatible when the target configuration provided these prerequisites. We also recorded tradeoffs and conditions that the literature left unresolved. The resulting applicability judgments form a conceptual synthesis because the publications did not evaluate every compatible pairing. Empirical evaluation remains necessary to establish effectiveness in a target configuration.

4 Related Work

Existing NAS-in-FL research addresses six broad purposes: coordinating federated search, reducing system costs, differentiating client architectures, protecting the search process, extending the optimization setting, and applies NAS in FL to specific domains.

General search-process research distributes candidate training and evaluation across clients while coordinating the search at the server [13, 40, 30, 15, 62, 75]. These publications establish complete NAS-in-FL methods that often combine several coordination mechanisms.

Resource-oriented research reduces search and deployment costs using methods tailored to computational, communication, and device constraints [65, 66, 47, 51, 73, 8, 63, 28, 36, 70, 64, 24, 3]. Their applicability depends on which cost is constrained.

Personalization research searches architectures at different levels of the federation and coordinates heterogeneous client models [69, 33, 72, 17, 27, 60, 58, 59, 61, 35]. This work separates the goal of differentiated architectures from the problem of coordinating them.

Security research uses privacy-preserving and robust mechanisms to protect search information and models [45, 67, 29, 38, 68, 71]. The available protection mechanisms depend on the representation exchanged during search.

Further work extends NAS-in-FL to broader objectives, model classes, and federated learning settings [44, 37, 42, 18, 52, 48, 49, 74, 19, 11, 32]. These extensions introduce assumptions that restrict transfer to other settings.

Application research evaluates NAS-in-FL across a range of specialized domains [22, 31, 39, 46, 56, 55, 4, 43, 26, 57, 54, 41, 25, 6, 7, 20]. It demonstrates domain breadth, although specialized evaluations provide limited evidence about mechanism reuse elsewhere.

4.1 Prior Syntheses of NAS-in-FL

One early review classifies complete methods by online and offline execution or by single- and multi-objective search [76]. Broader reviews compare NAS with hyperparameter optimization or place architecture search within multi-objective FL [23, 14]. These classifications organize procedures and objectives rather than the relationship between challenge and solution.

The most similar synthesis decomposes NAS-in-FL into design elements, method archetypes, traits, and suitable FL systems [16]. General NAS and FL reviews supply complementary concepts for search design and federated constraints [10, 1]. Their units remain complete methods, NAS components, or FL techniques.

4.2 Position of This Review

This thesis instead synthesizes recurring challenges and solution mechanisms. It separates solution mechanisms that are part of complete NAS-in-FL methods and maps their many-to-many relationships with challenges. This perspective supports assessing reuse under specified NAS design choices and FL system characteristics without claiming that unevaluated combinations will work.

5 Challenges and Solutions for Using NAS in FL

Our thematic analysis identified ten recurring challenges, organized as subthemes within five themes: *client resource constraints*, *client heterogeneity*, *decentralized search state*, *participation dynamics*, and *security*. For each challenge, we explain how the identified solution mechanisms address them. See Table 5.1 for an overview.

A challenge may have several solutions, and one solution may address several challenges. For clarity, each solution appears under the challenge it primarily addresses. Links to other challenges are identified as cross-cutting. We generally qualify challenges and solutions with the NAS method types of the publications they orignate from.

5.1 Client Resource Constraints

Training many candidate architectures from scratch makes NAS computationally demanding [53]. FL clients often have much less compute and memory available than their centralized counterparts. The challenges below deal with this difficulty.

Challenge 1: Client Capacity Limits the Search Work Assigned Per Round

NAS-in-FL becomes infeasible when an assigned search workload exceeds a client's compute or memory. Hardware differences compound this problem because the same workload may fit one client but not another [8, 66, 13, 59, 3]. Differentiable supernet-based strategies are particularly demanding because clients update several candidate operations within a shared supernet [15, 45, 42, 55]. Other strategies usually assign one candidate at a time, but even that candidate can exceed a low-end client's capacity. This challenge concerns whether clients can execute their assigned task. Challenge 9 concerns how long capable clients take to complete it.

Solution 1a: Use resource-efficient search components. Practitioners can restrict the search space to inexpensive components, such as depthwise-separable convolutions and mobile-oriented CNN blocks [55, 43]. This makes every candidate cheaper to train, but may exclude costly components that improve prediction accuracy. Solution 10a instead tests candidate modules on server data.

Solution 1b: Use capacity-compatible subnets or paths. Rather than make every client train the complete supernet, the search can assign a subnet or active path that fits the client's limits. Its depth, width, channel count or sampling budget can reflect available compute and memory [8, 47, 59, 63, 3, 61]. Single-path local search provides a related way to reduce memory use [13, 51, 26]. As a result, some operations or widths may receive fewer updates, as discussed in Challenge 6.

Solution 1c: Search trainable adaptation modules over a frozen backbone. When a suitable pretrained model is available, clients can keep its backbone fixed and search for a small combination of lightweight adaptation modules. Only these modules and their architecture parameters require local training and exchange, reducing computation and update size [52]. The benefit depends on how well the fixed backbone transfers to the clients' tasks.

Cross-cutting solution 7d. Architecture-agnostic distillation lets small client models teach a server-held supernet without requiring clients to run it [60].

Challenge 2: Architecture Search Adds Computational Cost

Even when clients can handle each round, the complete architecture search can take too long. Black-box strategies may train several candidates in separate federated evaluations [65, 75]. Weight-sharing strategies instead repeatedly update a large supernet or sampled subnets [15, 24]. Split-model execution can also leave clients idle while other segments run [19]. Some strategies discard the search-trained weights and retrain the selected architecture from a new initialization, adding more local epochs and aggregation rounds [57, 71]. This challenge arises from repeated candidate evaluation, idle resources or search work that produces no reusable weights. Challenge 3 addresses communication volume.

Solution 2a: Use multi-fidelity evaluation for candidate architectures. Black-box strategies can first evaluate each candidate with only a few communication rounds. Intermediate validation performance determines which candidates stop early and which receive more rounds. This multi-fidelity approach is used across several black-box search strategies and follows established AutoML methods [12].

For example, [65, 66] report that the best candidate usually becomes distinguishable within the first few rounds. Their procedures drop weaker candidates and increase the budget in later search iterations. Some evolutionary strategies likewise use a small fixed budget or early stopping [74, 49].

Solution 2b: Prune a supernet progressively. Supernet-based strategies can periodically remove weak operations, leaving fewer alternatives to store and train in later rounds [13, 59]. Unlike changing the evaluation budget, pruning reduces the search space itself.

Whether the final model needs separate training depends on how its weights are retained, as addressed by Solution 2e and Solution 2g. Pruning saves more work as search progresses, but early rounds still train the complete supernet and premature pruning can remove useful operations.

Solution 2c: Predict candidate performance. Performance prediction reduces costly configuration evaluations in AutoML [12]. NAS-in-FL strategies can evaluate a small set of architectures and train a predictor to estimate the quality of the rest [60, 46].

Because private, non-IID data can produce different rankings at each client, a separate predictor can be trained for each client [60]. Predictors can also be learned collaboratively without transmitting locally evaluated architectures [30]. This reduces client evaluations, but works only if the evaluated architectures represent the relevant search-space regions.

Solution 2d: Evaluate independent candidates or subnets in parallel. Black-box strategies can train and evaluate different candidates concurrently on separate client groups [65, 75, 46].

Supernet-based strategies can sample several subnets, assign each to a client group and merge their updates into the shared supernet [28]. This can reduce search time when enough clients are available, but different group data can bias candidate comparisons [65]. Challenge 6 addresses this problem.

Solution 2e: Reuse search-trained weights for candidate evaluation and deployment. In weight-sharing strategies, candidates can inherit the operation weights associated with their architecture [48, 70]. This avoids training every candidate from scratch and can initialize the selected subnet before limited fine-tuning [57]. Here, architecture selection remains separate from supernet training. Solution 2g instead produces the architecture and deployable weights together. Shared weights may still rank candidates poorly or provide a weak final initialization.

Solution 2f: Use split-training idle periods for local search. Temporary input and output components can make an assigned model segment executable on its own. During idle periods, a client can then sample and train alternative branches without waiting for the other model segments [19]. The assigned segment can also provide the feature-based teaching signal described in Solution 7d. This converts waiting time into search work.

Solution 2g: Jointly derive an architecture and its deployable weights. One-stage supernet strategies can train model weights while narrowing the architecture choices until the search produces a discrete model with trained weights [13, 18]. Reinforcement-learning strategies over a shared supernet can similarly use later rounds to fine-tune the final model as sampling concentrates on one path [61]. Unlike Solution 2b, this makes final model production part of search. The selected path may remain undertrained if competing paths received most updates.

Challenge 3: Architecture Search Increases Communication Costs

FL with a predefined architecture exchanges one model's parameters. NAS-in-FL may send larger updates, synchronize more often and exchange many candidate models. A supernet stores weights for alternative operations and can be much larger than the selected architecture. Differentiable strategies may also send architecture parameters throughout search [8, 60, 15, 4].

Solution 3a: Use compact codes to identify known candidates. When clients already hold a compatible master model or trained supernet, the server can send a compact selector or genotype instead of the candidate's parameters. Clients reconstruct the subnet locally and return its evaluation or updated weights [75, 57, 43]. Unlike Solution 7b, this requires a shared search space and common retained weights.

Solution 3b: Synchronize architecture parameters less frequently. Clients can update and upload architecture parameters less often than model weights. One approach transmits them every H_α rounds and stops once the architecture is fixed, while weight training continues [4].

Solution 3c: Aggregate search updates hierarchically. A nearby aggregator can combine updates from several clients before forwarding one result to the remote server. This shifts most aggregation to local links and reduces the search traffic transferred over the wider network [28].

The selected architecture also determines later communication costs. A model chosen only for predictive performance or local computation may have many parameters that FL must transmit in every training round. It can therefore meet compute and memory limits while exceeding the transfer budget [74]. This challenge covers communication during search and during later training of the selected model.

Cross-cutting solution 4b. This solution can include model size or expected transmitted bytes in candidate scores, allowing the search to consider later communication costs before selecting an architecture [74, 4].

5.2 Client Heterogeneity

Clients can differ substantially in compute, network capacity and data distributions. These differences affect which architectures are suitable and how candidate updates should be interpreted.

Challenge 4: No Single Architecture Meets All Deployment Requirements

A NAS-in-FL deployment may include devices with different inference-time compute, memory and latency limits. An architecture sized for the weakest client can underuse stronger hardware, while one sized for the strongest client may not run elsewhere [8]. The search must therefore produce architectures for several targets. Running an independent federated search for each target repeats computation and communication, so costs grow with the number of device groups or deployment budgets [24].

Solution 4a: Train one supernet for different deployment constraints. A weight-sharing supernet can train subnets with different depths, widths or exits in one federated process. Clients with similar limits can receive one architecture per device group [8, 69]. Finer-grained selection can instead choose a subnet for each client's hardware limits and validation data [24, 60, 72].

This solution selects the deployed model, while Solution 1b adjusts the workload a client executes during search. In [24], local predictor can select subnets without further federated training, reducing training cost by a factor of 11 for 20 targets in one evaluation, but training many subnets together can cause interference and leave rarely sampled parameters stale.

Solution 4b: Include resource costs in the architecture search objective. Supernet-based strategies can score predictive performance together with measured deployment costs. Latency or operation-count limits can tailor the result to each client group's hardware [69, 26, 63, 68]. The score can also penalize model size or expected bytes transmitted during later federated training [74, 4]. Unlike Solution 1a and Solution 10a, this changes how candidates are scored without removing them. The search can therefore balance prediction quality, inference limits and later communication before selecting an architecture.

Challenge 5: Data Heterogeneity Creates Conflicting Architecture Updates

Architecture updates depend on each client's data. Differences in class balance and features can make clients favor different operations or rank the same candidates differently [33, 24]. Clients solving related tasks may also require different outputs [17]. Architecture preferences can also vary over time as client data regimes change [4].

Solution 5a: Cluster clients by learned architecture preference. Clients with similar locally learned architecture parameters can form a group and train one architecture together [39]. Unlike grouping by latency or data balance, this mechanism groups clients by their learned architecture preferences.

Solution 5b: Optimize the worst-performing subnets across client groups. Instead of minimizing only average loss, the search can focus on the subnet with the highest loss in each client group. Sampling the smallest, largest and random subnets can approximate this objective [24]. Giving more attention to poorly performing client-subnet pairs prevents easier pairs from dominating shared-weight training.

Solution 5c: Split the architecture into shared and task-specific components. When clients solve related tasks, the search space can separate a shared feature extractor from task-specific outputs. Clients collaborate on the shared structure while selecting and fine-tuning local components for their task [17].

Cross-cutting solution 6b. Grouping clients to improve data coverage can make each candidate's evaluation better reflect the target population [28].

Challenge 6: Capability-Based Assignment Biases Candidate Evaluation

Assigning only subnets that fit each client's hardware allows more clients to contribute, but also determines which parts of the supernet they train. Operations and widths found only in large subnets may receive updates from few capable clients, while unselected parameters receive none [8, 59, 3]. Treating all parameters as equally trained can then distort the shared search state.

With non-IID data, assignment also ties each model size to the data of clients that can run it. Small candidates may train mainly on low-capability clients and large candidates on well-connected clients [8]. Candidate scores may then reflect the assignment pattern rather than architecture quality, even when every selected client returns an update [60]. Unequal training and biased data coverage are separate effects of the same assignment process.

Solution 6a: Aggregate each parameter only from clients that trained it. The server can aggregate an operation only from clients that updated it and weight their updates by the number of examples processed by that operation [8, 63, 3]. The resulting aggregation weights differ by parameter and reflect how much training each received. This avoids treating missing parameter updates as ordinary client contributions.

Solution 6b: Form groups based on resources and representative data. Client groups can meet a time or bandwidth limit while jointly representing the target class distribution. Methods either rebalance bandwidth-based groups by class distribution or minimize each group's difference from the population under a completion-time limit [36, 28]. Forming a group for each candidate makes comparisons less dependent on which clients can run it. The same approach can keep recurring client availability from biasing round-level participation.

Cross-cutting solution 7d. Architecture-agnostic distillation lets candidates learn from client predictions or features even when those clients cannot run the candidates [60, 33].

5.3 Decentralized Search State

NAS-in-FL distributes architecture parameters and model weights across clients and rounds. The resulting challenges concern whether these updates can be combined meaningfully and whether candidate evaluations remain comparable as the FL setup and architecture change.

Challenge 7: Distributed Search States Cannot Always Be Averaged

Differentiable NAS trains model weights and architecture parameters together in a shared supernet. The architecture parameters control which operations contribute, and their updates depend on the current operation weights. Aggregating only model weights loses local architecture decisions. Combining architecture parameters with unrelated weights separates those decisions from the state that produced them [15, 13, 42]. Direct aggregation therefore assumes that every client uses the same supernet and that its variables have the same meaning.

Independent local searches can return graphs with different connections, operations or blocks. Because their variables refer to different structures, ordinary averaging can lose architecture information [38, 51]. Aggregating only shared components also discards knowledge from components unique to a local architecture [73].

Solution 7a: Aggregate matching architecture parameters and model weights together. When clients use the same supernet, they can return both architecture parameters and model weights after local search. The server aggregates and redistributes them together [15, 13, 42]. This maintains one shared search state across rounds, but requires each variable to have the same meaning and shape at every client.

Solution 7b: Aggregate architecture encodings separately from model parameters. Clients can upload compact graph encodings without model weights. The server counts how often connections and operations occur and uses these frequencies to sample a global graph [38]. A related strategy assembles frequently recurring subgraphs directly [51]. Both preserve architecture information without matching all local model parameters. Compact encodings may contain fewer values than continuous gradients, but provide no formal privacy guarantee.

Solution 7c: Combine matching blocks based on distribution similarity. When different local architectures contain compatible blocks, the server can give more weight to blocks whose parameter distributions resemble that of the server block. Similarity is measured with Jensen–Shannon divergence [73]. This transfers parameters between matching blocks without assuming that the complete architectures match.

Solution 7d: Transfer knowledge between architectures through distillation. Client predictions or feature maps on shared inputs can teach a server supernet, sampled subnets or different blocks. A target architecture can therefore learn from clients that cannot run it or whose parameters do not match [73, 60, 33]. Compressed predictions can likewise support collaboration between entirely different architectures [29]. This avoids matching parameters directly, but requires shared inputs and transfers model behavior rather than complete parameters.

Challenge 8: Architecture Evaluation Depends on the FL Configuration

Because raw client data remain local, candidates cannot usually be trained and validated on the same pooled data [21]. Their scores instead depend on the FL setup, including the number of local epochs and the fraction of participating clients. An architecture selected under one setup may rank differently under another [42, 44, 22]. Changing these settings after search can therefore invalidate the earlier comparison.

Solution 8a: Search architectures and FL settings together. An alternating procedure can choose a training setup, search an architecture under it and use the result to choose the next setup [42]. Evolutionary and black-box strategies can instead search the architecture together with settings such as local epochs and client participation, then evaluate each combination through federated training [44, 22]. Candidate scores then reflect the FL procedure in which the architecture will operate.

Candidate scores can also change with the current search state. Layer-wise updates may favor different operations at different depths, and an early preference for operations without parameters can prevent convergence [25]. Fixing one operation changes the computation graph and weight sharing, so the next choice may be evaluated before the remaining weights adapt [35]. Changing which subnets are trained can likewise make aggregated updates unstable and slow convergence [47]. A score can therefore reflect when and how a candidate was evaluated, not only its architecture.

Solution 8b: Regularize layer-wise architecture updates. During local differentiable search, an added term can align layer gradients with architecture parameters. This reduces depth-dependent preferences for particular operations before client updates are aggregated [25].

Solution 8c: Insert recovery rounds between architecture decisions. A progressive supernet search can first warm up the shared weights and then fix one operation at a time based on local validation accuracy. After each decision, several recovery rounds update the remaining weights before the next edge is evaluated [35]. Unlike Solution 2b, this pause helps weights adapt to each architecture change. Once all relevant edges are resolved, ordinary training continues with a fixed architecture.

Solution 8d: Vary which subnets are trained over time. The server can assign very different subnets in early rounds and increasingly similar ones as training advances. Early rounds explore separate parts of the search space, while later rounds concentrate on better understood regions [47]. These structured assignments also combine more consistently than independently sampled subnets.

5.4 Participation Dynamics

The participating clients and the timing of their updates can change throughout architecture search. Clients may disconnect, respond at different speeds, converge at different times or participate with different frequencies.

Challenge 9: Variable Client Participation Disrupts Search

Clients can become unavailable because of changing network conditions, battery limits or competing local work. A search that requires the same clients in every round may stall or lose updates for candidates or supernet paths assigned to disconnected clients [70].

Available clients may still complete search work at different speeds. A round may wait for the slowest client, while an update that arrives after the global search has advanced may already be outdated [19, 62, 4]. This differs from resource infeasibility, where a client cannot run the workload, and dropout, where it returns no update.

Solution 9a: Group split-model segments by training time. Split-model strategies can measure client computation and communication times and group segments that finish at similar times. Unlike Solution 6b, this groups clients only by observed timing so that supernet training waits less for the slowest segment [19].

Solution 9b: Adjust the influence of outdated search updates. The server can account for delay when adding a late update after the global search has advanced [62]. Differentiable search can instead give older updates less weight [4]. Unlike Solution 6a, these approaches adjust an update according to its age rather than how much each parameter was trained.

In personalized differentiable search, clients can learn architecture parameters at different rates because their data and local training differ. One shared stopping round can end some searches too early while allowing others to overfit, for example by increasingly favoring skip connections [33]. Clients can therefore need different stopping times even when their updates arrive promptly.

Solution 9c: Stop architecture updates independently. After a shared warm-up period, each client can stop updating its architecture when a local test detects growing preference for skip connections. Other clients continue until they meet their own stopping condition [33].

Partial participation and random client selection can cause some clients to contribute more often than others [70, 59]. Even if the search continues, frequently available clients influence shared weights and architecture rankings more often. The result may therefore poorly represent clients that participate less often.

Cross-cutting solution 6b. This solution can select participants by both resources and data coverage so that availability alone does not determine whose data enter the search [36, 28].

5.5 Search Security

NAS-in-FL communicates information about the architecture in addition to conventional model updates. Malicious participants can interfere with the search, while updates from honest clients may reveal private information.

Challenge 10: Architecture Search Introduces New Security Risks

NAS-in-FL creates additional attack targets because clients influence both model weights and search signals that determine the selected architecture. Malicious clients can manipulate either through poisoned data or local training, steering the search toward a model that supports a backdoor attack. Weight sharing can spread poisoned weights across candidates. Fourfold-scaled updates from one of eight clients degraded two discrete strategies, showing that poisoning can persist in the selected architecture [71].

Conditional observation. During differentiable supernet-based search, each candidate operation remains in a weighted mixture while clients update its weights and selection probability. [71] finds that an operation contributes less when its poisoned weights work against or contribute little to the main task.

[71] demonstrates this behavior only for the studied backdoor attacks and search spaces. Retraining the selected fixed architecture from scratch did not preserve its resistance to attack, suggesting that the effect belongs to the trained search state rather than the architecture itself.

Solution 10a: Pre-screen search-space modules on server data. The server can use its own data to select a compact set of accurate, efficient modules before federated search. In [71], this reduced complexity and limited overfitting to noise from backdoor triggers. Unlike Solution 1a, the mechanism tests modules using data. Unlike Solution 4b, it removes modules before search instead of changing their scores.

NAS-in-FL transmits data-dependent architecture gradients and rewards that fixed-architecture FL does not [45, 42]. Other strategies transmit structural encodings or personalized choices that may reveal characteristics of client data, individual examples or which client sent an update, although [38, 71] does not demonstrate such attacks. The risk depends on what is transmitted and the attacker's capabilities.

Solution 10b: Apply differential privacy to search signals. Differentiable strategies can limit the size of model and architecture gradients and add Gaussian noise before upload. Noise can also be added to rewards used for hyperparameter selection [45, 42]. In vertically partitioned search, the intermediate activations and backward gradients exchanged between parties can receive the same protection [32]. The noise can, however, reduce model and search performance [71].

Solution 10c: Apply secure aggregation to matching search variables. When every client updates the same differentiable supernet, model weights and architecture parameters have matching structures and can use the same secure-aggregation protocol [71]. This hides each client's updates from the server but requires every client to use the same search representation.

Cross-cutting solution 7b. The compact structural encoding exchanged in this solution may make it harder to reconstruct inputs because it has far fewer dimensions than the private data. The cited analysis argues that a small derivative matrix can correspond to many image batches, making exact pixel recovery by that procedure impractical [38]. However, a compact encoding provides no formal privacy guarantee. It may still reveal properties of the client data or the source of an update and may require differential privacy or encryption.

5.6 Overview

Table 5.1 summarizes the final set of synthesized challenges and solutions. Cross-cutting entries indicate where a solution primarily discussed under another challenge also applies.

Table 5.1: Overview of the identified challenges, their primary solutions, and cross-cutting applications.

Theme	Challenge	Solution	Example publication
Client resource constraints	Client Capacity Limits the Search Work Assigned Per Round	Use resource-efficient search components	[55]
		Use capacity-compatible subnets or paths	[8]
		Search trainable adaptation modules over a frozen backbone	[52]
		Cross-cutting: Transfer knowledge between architectures through distillation	[60]
	Architecture Search Adds Computational Cost	Use multi-fidelity evaluation for candidate architectures	[65]
		Prune a supernet progressively	[13]
		Predict candidate performance	[60]
		Evaluate independent candidates or subnets in parallel	[65]
		Reuse search-trained weights for candidate evaluation and deployment	[57]
		Use split-training idle periods for local search	[19]
		Jointly derive an architecture and its deployable weights	[13]
	Architecture Search Increases Communication Costs	Use compact codes to identify known candidates	[75]
		Synchronize architecture parameters less frequently	[4]
		Aggregate search updates hierarchically	[28]
		Cross-cutting: Include resource costs in the architecture search objective	[74]
Client heterogeneity	No Single Architecture Meets All Deployment Requirements	Train one supernet for different deployment constraints	[24]
		Include resource costs in the architecture search objective	[69]
	Data Heterogeneity Creates Conflicting Architecture Updates	Cluster clients by learned architecture preference	[39]
		Optimize the worst-performing subnets across client groups	[24]
		Split the architecture into shared and task-specific components	[17]
		Cross-cutting: Form groups based on resources and representative data	[28]

Table 5.1 continued

Theme	Challenge	Solution	Example publication
	Capability-Based Assignment Biases Candidate Evaluation	Aggregate each parameter only from clients that trained it	[8]
		Form groups based on resources and representative data	[28]
		Cross-cutting: Transfer knowledge between architectures through distillation	[60]
Decentralized search state	Distributed Search States Cannot Always Be Averaged	Aggregate matching architecture parameters and model weights together	[15]
		Aggregate architecture encodings separately from model parameters	[38]
		Combine matching blocks based on distribution similarity	[73]
		Transfer knowledge between architectures through distillation	[73]
	Architecture Evaluation Depends on the FL Configuration	Search architectures and FL settings together	[42]
		Regularize layer-wise architecture updates	[25]
		Insert recovery rounds between architecture decisions	[35]
		Vary which subnets are trained over time	[47]
Participation dynamics	Variable Client Participation Disrupts Search	Group split-model segments by training time	[19]
		Adjust the influence of outdated search updates	[62, 4]
		Stop architecture updates independently	[33]
		Cross-cutting: Form groups based on resources and representative data	[36]
Search security	Architecture Search Introduces New Security Risks	Pre-screen search-space modules on server data	[71]
		Apply differential privacy to search signals	[45]
		Apply secure aggregation to matching search variables	[71]
		Cross-cutting: Aggregate architecture encodings separately from model parameters	[38]

6 Discussion

This chapter interprets the relationship between challenges and solutions, explains their implications for reusing solution mechanisms, and qualifies the conclusions supported by the reviewed literature.

6.1 Principal Findings

The analysis identified ten challenges across five themes and 34 solution mechanisms. Table 5.1 maps the relationships between these mechanisms and challenges. The analysis further indicates that a mechanism is conceptually reusable when the target NAS design choices and FL system characteristics satisfy its prerequisites and it remains compatible with other mechanisms in the configuration.

Challenge relevance depends on which search functions rely on clients. Client-side search workloads expose the process to resource constraints, distributed candidate evaluation makes evidence dependent on local data, and decentralized search-state updates require coordination across participants. Retaining a function at the server can avoid the corresponding client-side constraint, but may require representative server data or greater trust in the server. The challenges consequently arise from specific distributed dependencies rather than from decentralization as one uniform property.

Solution mechanisms also interact. Assigning capacity-compatible subnets reduces client workloads, but causes operations available only to capable clients to receive fewer updates. Exposure-aware aggregation is then needed to prevent unequal training from distorting the shared state [8, 63, 3]. Similarly, specialized architectures can accommodate different deployment requirements while removing the parameter alignment required for ordinary aggregation. Structural aggregation or distillation can restore collaboration under this condition [38, 73, 60]. A solution should therefore be assessed as part of a configuration rather than as an independent remedy.

The search space creates a recurring trade-off between architectural coverage and coordination. Restricting it to inexpensive, structurally aligned components lowers client workloads and enables direct aggregation, but may exclude architectures that better fit particular data or devices. Elastic or personalized spaces cover more deployment conditions, but increase search effort and can produce unequal parameter exposure or incompatible local states. The search space thus serves both as the set of possible architectures and as a coordination contract between clients.

Brief training is the predominant performance-estimation strategy in the reviewed literature. This common choice means that challenge applicability is often determined more directly by the workload assigned to clients, the relationship between their search states, and whether the search produces a shared or specialized architecture. Broad search-strategy categories alone can conceal these differences. Reasoning about reuse therefore requires the more specific NAS design choices and FL system characteristics retained in Chapter 5.

6.2 Implications for Practice and Research

The synthesis provides a diagnostic process for selecting solution mechanisms. Practitioners can first characterize the intended FL system and NAS method type, identify the challenges activated by this combination, and then select mechanisms whose prerequisites are satisfied. They must subsequently check whether the chosen mechanisms introduce additional costs or incompatible representations. The tables support this process, but they neither rank mechanisms nor guarantee that combinations not evaluated together will be effective.

For research, the synthesis complements method-level taxonomies by treating challenges and solution mechanisms as separate units. This exposes relationships that remain hidden when several mechanisms are presented as one NAS-in-FL method. It also provides a common vocabulary for comparing mechanisms under stated conditions. Future publications can support such comparisons by reporting the FL system characteristics, client workloads, exchanged search state, search space, and performance-estimation strategy used in their evaluations.

6.3 Limitations

The review depends on what the selected publications report. It can include only challenges, solution mechanisms, and applicability conditions described with enough detail to meet the inclusion criteria. Operational problems that have not been published or that receive only general descriptions may therefore be absent. The Scopus search and citation searching reduce this risk but do not establish that the corpus covers every relevant publication.

The thematic analysis also requires judgment when concepts are named, separated, merged, and mapped [2]. Explicit definitions and an audit trail make these decisions transparent, but do not eliminate interpretive subjectivity. Moreover, few publications isolate the effect of one FL characteristic or solution mechanism. The resulting mappings represent reported relationships and mechanism-based applicability judgments, not independent estimates of causal effects or comparative effectiveness.

Evaluations differ in their datasets, search spaces, client populations, participation, resource models, and training budgets. These differences prevent a quantitative ranking of the mechanisms. Incomplete descriptions of FL systems further limit transfer judgments because an unreported condition cannot be distinguished from one that was absent. Publication frequency therefore indicates attention within the corpus, not practical prevalence, severity, or solution quality.

6.4 Future Research

Future evaluations should test individual mechanisms and selected combinations under shared configurations. Varying one FL condition or NAS property at a time would provide stronger evidence for the relationships identified in this review. Such evaluations should separate search computation, final-model training, communication, wall-clock time, candidate-ranking fidelity, and final architecture quality.

Scale and participation require particular attention. Experiments should vary the number of clients together with partial participation, dropout, latency, and capability-dependent selection, since a larger simulated population alone does not reproduce cross-device dynamics. Publications should also distinguish costs measured on client hardware from those simulated on centralized infrastructure.

A shared reporting scheme would make results easier to compare. At minimum, publications should describe client resources, network conditions, data distribution, participation, federation size, trust assumptions, client search workloads, exchanged search state, search space, and final-model development. Research should also examine sparsely studied areas of the challenge–solution map and test whether low-cost performance estimators from centralized NAS remain reliable when candidate evidence is distributed across heterogeneous client data [16].

7 Conclusion

Solution mechanisms identified in the NAS-in-FL literature can be reused at a conceptual level when the target NAS design choices and FL system characteristics satisfy their prerequisites. Through a concept-centric literature review and thematic analysis, this thesis identified ten challenges across five themes and 34 solution mechanisms. Table 5.1 connects these mechanisms to the challenges they address. The analysis further delimits the NAS method types and FL system characteristics with which they are compatible. Empirical evaluation remains necessary to establish the effectiveness of a mechanism in a compatible configuration.

Separating solution mechanisms from the complete NAS-in-FL methods in which they appear makes recurring design components visible across different terminology and implementations. Researchers and practitioners can characterize a target configuration, identify the challenges created by its distributed dependencies, and examine mechanisms whose required structures and operations remain available. This synthesis provides a structured basis for comparing and designing NAS-in-FL methods, while comparative evaluations are required to rank candidate mechanisms.

The central conclusion is that reuse is context-dependent. NAS design choices and FL system characteristics jointly determine which challenges arise, and solution mechanisms can introduce prerequisites, costs, or interactions with other mechanisms. The reviewed evaluations cover uneven configurations and often describe their FL systems incompletely, which restricts judgments about transfer to other configurations. Progress therefore depends on studying mechanisms as parts of explicitly specified configurations.

Future research should evaluate individual mechanisms and selected combinations under the same NAS design choices and FL system characteristics. These evaluations should represent operational conditions such as scale and participation and should report end-to-end costs alongside architecture quality. Accumulating evidence at the mechanism level will enable NAS-in-FL research to establish which reusable design components work under which conditions.

Abbreviations

List of Figures

List of Tables

Bibliography

[1] M. Arbaoui, M.-A. Brahmia, A. Rahmoun, and M. Zghal. "Federated learning survey: A multi-level taxonomy of aggregation techniques, experimental insights, and future frontiers." In: *ACM Trans. Intell. Syst. Technol.* 15.6 (Nov. 2024). ISSN: 2157-6904. DOI: 10.1145/3678182.

[2] V. Braun and V. Clarke. "Using thematic analysis in psychology." In: *Qualitative Research in Psychology* 3.2 (2006), pp. 77–101. DOI: 10.1191/1478088706qp063oa. eprint: https://doi.org/10.1191/1478088706qp063oa.

[3] Y. Chen, D. Chen, and C. Zhong. "A supernet-only framework for federated learning in computationally heterogeneous scenarios." In: *Applied Sciences* 15.10 (2025). ISSN: 2076-3417. DOI: 10.3390/app15105666.

[4] Z. Chen, H. Zhang, S. Wang, and J. Chen. "FedRegNAS: Regime-Aware Federated Neural Architecture Search for Privacy-Preserving Stock Price Forecasting." In: *Electronics* 14.24 (2025). DOI: 10.3390/electronics14244902.

[5] K. Daly, H. Eichner, P. Kairouz, H. B. McMahan, D. Ramage, and Z. Xu. "Federated learning in practice: Reflections and projections." In: *2024 IEEE 6th international conference on trust, privacy and security in intelligent systems, and applications (TPS-ISA)*. 2024, pp. 148–156. DOI: 10.1109/TPS-ISA62245.2024.00026.

[6] A. Dolatabadi, H. Abdeltawab, and Y. A.-R. I. Mohamed. "SFNAS-DDPG: A biomass-based energy hub dynamic scheduling approach via connecting supervised federated neural architecture search and deep deterministic policy gradient." In: *IEEE Access* 12 (2024), pp. 7674–7688. DOI: 10.1109/ACCESS.2024.3352032.

[7] A. Dolatabadi, J. Chakravorty, and X. Feng. "Supervised federated neural architecture search and its application in power system forecasting." In: *2023 IEEE power & energy society general meeting (PESGM)*. 2023, pp. 1–5. DOI: 10.1109/PESGM52003.2023.10252675.

[8] L. Dudziak, S. Laskaridis, and J. Fernandez-Marques. *FedorAS: Federated architecture search under system heterogeneity*. 2022. arXiv: 2206.11239 [cs.LG].

[9] Elsevier. *Scopus*. Abstract and citation database. Accessed 2026-01-17. 2026.

[10] T. Elsken, J. H. Metzen, and F. Hutter. "Neural architecture search: A survey." In: *Journal of Machine Learning Research* 20.55 (2019), pp. 1–21.

[11] H. Fang, Y. Gao, P. Zhang, J. Yao, H. Chen, and H. Wang. "Large language models enhanced personalized graph neural architecture search in federated learning." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 39.16 (Apr. 2025), pp. 16514–16522. DOI: 10.1609/aaai.v39i16.33814.

[12] M. Feurer and F. Hutter. "Hyperparameter optimization." In: *Automated machine learning: Methods, systems, challenges*. Ed. by F. Hutter, L. Kotthoff, and J. Vanschoren. Cham: Springer International Publishing, 2019, pp. 3–33. ISBN: 978-3-030-05318-5. DOI: 10.1007/978-3-030-05318-5_1.

[13] A. Garg, A. K. Saha, and D. Dutta. *Direct federated neural architecture search*. 2020. arXiv: 2010.06223 [cs.LG].

[14] M. Hartmann, G. Danoy, and P. Bouvry. *Multi-objective methods in Federated Learning: A survey and taxonomy*. 2025. arXiv: `2502.03108 [cs.LG]`.

[15] C. He, M. Annavaram, and S. Avestimehr. *Towards Non-I.I.D. and invisible data with FedNAS: Federated deep learning via neural architecture search*. 2021. arXiv: `2004.08546 [cs.LG]`.

[16] N. Henze, S. Rank, D. Deng, M. Lindauer, A. Sunyaev, and N. Kannengießer. "Neural Architecture Search in Federated Learning: A Unified Design Concept." In: *TechRxiv* (2026). DOI: `10.36227/techrxiv.176964091.16274890/v1`.

[17] M. Hoang and C. Kingsford. "Personalized Neural Architecture Search for Federated Learning." In: *1st NeurIPS Workshop on New Frontiers in Federated Learning (NFFL 2021)* ().

[18] X. Huang and J. Gao. *Federated neural architecture search with model-agnostic meta learning*. 2025. arXiv: `2504.06457 [cs.LG]`.

[19] C. Huo, J. Jia, T. Deng, M. Dong, Z. Yu, and D. Yuan. "NASFLY: On-device split federated learning with neural architecture search." In: *2024 IEEE international symposium on parallel and distributed processing with applications (ISPA)*. 2024, pp. 2184–2190. DOI: `10.1109/ISPA63168.2024.00298`.

[20] H. Jin, K. Zhang, S. Fan, H. Jin, and B. Wang. "Wind power forecasting based on ensemble deep learning with surrogate-assisted evolutionary neural architecture search and many-objective federated learning." In: *Energy* 308 (2024), p. 133023. ISSN: 0360-5442. DOI: `https://doi.org/10.1016/j.energy.2024.133023`.

[21] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao. *Advances and Open Problems in Federated Learning*. 2021. arXiv: `1912.04977 [cs.LG]`.

[22] S. Khan, F. Nosheen, S. S. A. Naqvi, H. Jamil, M. Faseeh, M. A. Khan, and D.-H. Kim. "Bilevel Hyperparameter Optimization and Neural Architecture Search for Enhanced Breast Cancer Detection in Smart Hospitals Interconnected With Decentralized Federated Learning Environment." In: *IEEE Access* 12 (2024), pp. 63618–63628. DOI: `10.1109/ACCESS.2024.3392572`.

[23] S. Khan, A. Rizwan, A. N. Khan, M. Ali, R. Ahmed, and D. H. Kim. "A multi-perspective revisit to the optimization methods of Neural Architecture Search and Hyper-parameter optimization for non-federated and federated learning environments." In: *Computers and Electrical Engineering* 110 (2023), p. 108867. ISSN: 0045-7906. DOI: `https://doi.org/10.1016/j.compeleceng.2023.108867`.

[24] A. Khare, A. Agrawal, A. Annavajjala, P. Behnam, M. Lee, H. Latapie, and A. Tumanov. *SuperFedNAS: Cost-efficient federated neural architecture search for on-device inference*. 2024. arXiv: `2301.10879 [cs.LG]`.

[25] J. Li, S. Wang, R. Yang, M. Gong, Z. Hu, N. Zhang, K. Sheng, and Y. Zhou. "Toward Federated Customized Neural Architecture Search for Remote Sensing Scene Classification." In: *IEEE Transactions on Geoscience and Remote Sensing* 63 (2025), pp. 1–12. DOI: `10.1109/TGRS.2025.3537085`.

[26] Y. Li, R. Yao, D. Qin, and Y. Wang. "Lightweight federated learning for on-device non-intrusive load monitoring." In: *IEEE Transactions on Smart Grid* 16.2 (2025), pp. 1950–1961. DOI: `10.1109/TSG.2024.3482363`.

[27] O. Litany, H. Maron, D. Acuna, J. Kautz, G. Chechik, and S. Fidler. *Federated learning with heterogeneous architectures using graph HyperNetworks*. 2022. arXiv: `2201.08459 [cs.LG]`.

[28] J. Liu, J. Yan, H. Xu, Z. Wang, J. Huang, and Y. Xu. "Finch: Enhancing federated learning with hierarchical neural architecture search." In: *IEEE Transactions on Mobile Computing* 23.5 (2024), pp. 6012–6026. DOI: 10.1109/TMC.2023.3315451.

[29] J. Liu, K. Huang, and L. Xie. "HeteFed: Heterogeneous federated learning with privacy-preserving binary low-rank matrix decomposition method." In: *2023 26th international conference on computer supported cooperative work in design (CSCWD)*. 2023, pp. 1238–1244. DOI: 10.1109/CSCWD57460.2023.10152714.

[30] S. Liu, X. Wang, and Y. Jin. "Federated Bayesian Optimization for Privacy-Preserving Neural Architecture Search." In: *2023 IEEE Congress on Evolutionary Computation (CEC)*. 2023, pp. 1–8. DOI: 10.1109/CEC53210.2023.10254155.

[31] X. Liu, J. Zhao, J. Li, B. Cao, and Z. Lv. "Federated neural architecture search for medical data security." In: *IEEE Transactions on Industrial Informatics* 18.8 (2022), pp. 5628–5636. DOI: 10.1109/TII.2022.3144016.

[32] Y. Liu, X. Liang, J. Luo, Y. He, T. Chen, Q. Yao, and Q. Yang. "Cross-Silo Federated Neural Architecture Search for Heterogeneous and Cooperative Systems." In: *Federated and Transfer Learning*. Ed. by R. Razavi-Far, B. Wang, M. E. Taylor, and Q. Yang. Cham: Springer International Publishing, 2023, pp. 57–86. ISBN: 978-3-031-11748-0. DOI: 10.1007/978-3-031-11748-0_4.

[33] Y. Liu, S. Guo, J. Zhang, Z. Hong, Y. Zhan, and Q. Zhou. "Collaborative Neural Architecture Search for Personalized Federated Learning." In: *IEEE Transactions on Computers* 74.1 (2025), pp. 250–262. DOI: 10.1109/TC.2024.3477945.

[34] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. "Communication-efficient learning of deep networks from decentralized data." In: *Proceedings of the 20th international conference on artificial intelligence and statistics*. Ed. by S. Aarti and Z. Jerry. Vol. 54. Proceedings of machine learning research. PMLR, Apr. 2017, pp. 1273–1282.

[35] E. Mushtaq, C. He, J. Ding, and S. Avestimehr. *SPIDER: Searching personalized neural architecture for federated learning*. 2021. arXiv: 2112.13939 [cs.LG].

[36] G. Öcal and A. Özgövde. "Network-aware federated neural architecture search." In: *Future Generation Computer Systems* 162 (2025), p. 107475. ISSN: 0167-739X. DOI: https://doi.org/10.1016/j.future.2024.07.053.

[37] W. Pan, J. Li, L. Gao, S. Bao, Q. Zhao, Q. Wang, and C. Cui. "APONS: Accelerating federated learning architecture in 6G based on parameter optimization and neural architecture search." In: *Proceedings of the 2023 11th international conference on computer and communications management*. ICCCM '23. Nagoya, Japan: Association for Computing Machinery, 2023, pp. 194–199. ISBN: 9798400707735. DOI: 10.1145/3617733.3617764.

[38] Z. Pan, L. Hu, W. Tang, J. Li, Y. He, and Z. Liu. "Privacy-preserving multi-granular federated neural architecture search – a general framework." In: *IEEE Transactions on Knowledge and Data Engineering* 35.3 (2023), pp. 2975–2986. DOI: 10.1109/TKDE.2021.3116248.

[39] D. Qin, C. Wang, Q. Wen, W. Chen, L. Sun, and Y. Wang. "Personalized federated DARTS for electricity load forecasting of individual buildings." In: *IEEE Transactions on Smart Grid* 14.6 (2023), pp. 4888–4901. DOI: 10.1109/TSG.2023.3253855.

[40] T. Ramírez-Gordillo, H. Mora, F. A. Pujol, and A. Maciá-Lillo. "Federated Neural Architecture Search for Efficient and Privacy-Preserving Model Training." In: *2024 IEEE International Conference on Smart Internet of Things (SmartIoT)*. 2024, pp. 129–136. DOI: 10.1109/SmartIoT62235.2024.00028.

[41] H. R. Roth, D. Yang, W. Li, A. Myronenko, W. Zhu, Z. Xu, X. Wang, and D. Xu. "Federated whole prostate segmentation in MRI with personalized neural architectures." In: *Medical image computing and computer assisted intervention – MICCAI 2021*. Ed. by M. de Bruijne, P. C. Cattin, S. Cotin, N. Padoy, S. Speidel, Y. Zheng, and C. Essert. Cham: Springer International Publishing, 2021, pp. 357–366. ISBN: 978-3-030-87199-4.

[42] J. Seng, P. Prasad, M. Mundt, D. S. Dhami, and K. Kersting. *FEATHERS: Federated Architecture and Hyperparameter Search*. 2023. arXiv: 2206.12342 [cs.LG].

[43] J.-M. Shao, G.-Q. Zeng, K.-D. Lu, G.-G. Geng, and J. Weng. "Automated federated learning for intrusion detection of industrial control systems based on evolutionary neural architecture search." In: *Computers & Security* 143 (2024), p. 103910. DOI: 10.1016/j.cose.2024.103910.

[44] R. E. Shawi. "AgingFedNAS: Aging evolution federated deep learning for architecture and hyperparameter search." In: *Advances in knowledge discovery and data mining*. Ed. by X. Wu, M. Spiliopoulou, C. Wang, V. Kumar, L. Cao, Y. Wu, Y. Yao, and Z. Wu. Singapore: Springer Nature Singapore, 2025, pp. 107–119. DOI: 10.1007/978-981-96-8173-0_9.

[45] I. Singh, H. Zhou, K. Yang, M. Ding, B. Lin, and P. Xie. *Differentially-private federated neural architecture search*. 2020. arXiv: 2006.10559 [cs.LG].

[46] J. Su, C. Tang, and Z. Liu. "A Prediction Optimization Method with Federated Learning and Neural Architecture Search for Distributed Renewable Energy Sources." In: *Distributed Generation & Alternative Energy Journal* 40.2 (2025), pp. 213–238. DOI: 10.13052/dgaej2156-3306.4021.

[47] Y. Venkatesha, Y. Kim, H. Park, and P. Panda. "Divide-and-conquer the NAS puzzle in resource-constrained federated learning systems." In: *Neural Networks* 168 (2023), pp. 569–579. ISSN: 0893-6080. DOI: https://doi.org/10.1016/j.neunet.2023.10.006.

[48] C. Wang, B. Chen, G. Li, and H. Wang. "Automated Graph Neural Network Search under Federated Learning Framework." In: *IEEE Transactions on Knowledge and Data Engineering* 35.10 (2023), pp. 9959–9972. DOI: 10.1109/TKDE.2023.3250264.

[49] D. Wang. "Federated learning using GPT-4 boosted particle swarm optimization for compact neural architecture search." In: *Journal of Advances in Information Technology* 15.9 (2024). Cited by: 5; All Open Access, Gold Open Access, pp. 1011–1018. DOI: 10.12720/jait.15.9.1011-1018.

[50] J. Webster and R. T. Watson. "Analyzing the past to prepare for the future: Writing a literature review." In: *MIS Quarterly* 26.2 (2002), pp. xiii–xxiii. ISSN: 02767783.

[51] X. Wei, G. Chen, C. Yang, H. Zhao, C. Wang, and H. Yue. "EFNAS: Efficient federated neural architecture search across AIoT devices." In: *2024 international joint conference on neural networks (IJCNN)*. 2024, pp. 1–8. DOI: 10.1109/IJCNN60899.2024.10650653.

[52] X. Wen and S. Xu. "When neural network architecture search meets federated learning parameter efficient fine tuning." In: *2023 3rd international conference on intelligent communications and computing (ICC)*. 2023, pp. 180–185. DOI: 10.1109/ICC59986.2023.10421429.

[53] C. White, M. Safari, R. Sukthanker, B. Ru, T. Elsken, A. Zela, D. Dey, and F. Hutter. *Neural architecture search: Insights from 1000 papers*. 2023. arXiv: 2301.08727 [cs.LG].

[54] R. Wu, C. Li, J. Zou, X. Liu, H. Zheng, and S. Wang. "Generalizable reconstruction for accelerating MR imaging via federated learning with neural architecture search." In: *IEEE Transactions on Medical Imaging* 44.1 (2025), pp. 106–117. DOI: 10.1109/TMI.2024.3432388.

[55] R. Wu, C. Li, J. Zou, and S. Wang. "FedAutoMRI: Federated neural architecture search for MR image reconstruction." In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023 Workshops*. Berlin, Heidelberg: Springer, 2023, pp. 347–356. DOI: 10.1007/978-3-031-47401-9_33.

[56] X. Wu, Y.-T. Zhang, K.-W. Lai, M.-Z. Yang, G.-L. Yang, and H.-H. Wang. "A novel centralized federated deep fuzzy neural network with multi-objectives neural architecture search for epistatic detection." In: *IEEE Transactions on Fuzzy Systems* 33.1 (2025), pp. 94–107. DOI: 10.1109/TFUZZ.2024.3369944.

[57] Y. Wu, H. Fan, W. Ying, Z. Zhou, Q. Zheng, and J. Zhang. "Federated Neural Architecture Search with Hierarchical Progressive Acceleration for Medical Image Segmentation." In: *Advances in Swarm Intelligence*. Ed. by Y. Tan and Y. Shi. Singapore: Springer Nature Singapore, 2024, pp. 112–123. ISBN: 978-981-97-7184-4.

[58] Z. Xu, L. Yang, and S. Gu. "Heterogeneous federated learning based on graph hypernetwork." In: *Artificial neural networks and machine learning – ICANN 2023*. Ed. by L. Iliadis, A. Papaleonidas, P. Angelov, and C. Jayne. Cham: Springer Nature Switzerland, 2023, pp. 464–476. ISBN: 978-3-031-44213-1.

[59] J. Yan, J. Liu, H. Xu, Z. Wang, and C. Qiao. "Peaches: Personalized federated learning with neural architecture search in edge computing." In: *IEEE Transactions on Mobile Computing* 23.11 (2024), pp. 10296–10312. DOI: 10.1109/TMC.2024.3373506.

[60] A. Yang and Y. Liu. "Heterogeneity-aware personalized federated neural architecture search." In: *Entropy* 27.7 (2025). ISSN: 1099-4300. DOI: 10.3390/e27070759.

[61] D. Yao and B. Li. "PerFedRLNAS: One-for-all personalized federated neural architecture search." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 38.15 (Mar. 2024), pp. 16398–16406. DOI: 10.1609/aaai.v38i15.29576.

[62] D. Yao, L. Wang, J. Xu, L. Xiang, S. Shao, Y. Chen, and Y. Tong. "Federated model search via reinforcement learning." In: *2021 IEEE 41st international conference on distributed computing systems (ICDCS)*. 2021, pp. 830–840. DOI: 10.1109/ICDCS51616.2021.00084.

[63] W. Ying, C. Wang, Y. Wu, X. Luo, Z. Wen, and H. Zhang. "FGFL: Fine-grained federated learning based on neural architecture search for heterogeneous clients." In: *Advances in Swarm Intelligence*. Singapore: Springer Nature Singapore, 2024, pp. 99–111.

[64] S. Yu, J. P. Muñoz, and A. Jannesari. "Resource-aware heterogeneous federated learning with specialized local models." In: *Euro-par 2024: Parallel processing: 30th european conference on parallel and distributed processing, madrid, spain, august 26–30, 2024, proceedings, part I*. Madrid, Spain: Springer-Verlag, 2024, pp. 389–403. ISBN: 978-3-031-69576-6. DOI: 10.1007/978-3-031-69577-3_27.

[65] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang. *Federated neural architecture search*. 2022. arXiv: 2002.06352 [cs.LG].

[66] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang. "Resource-Aware Federated Neural Architecture Search over Heterogeneous Mobile Devices." In: *IEEE Transactions on Big Data* (2022). DOI: 10.1109/TBDATA.2022.3227403.

[67] G.-Q. Zeng, J.-M. Shao, K.-D. Lu, G.-G. Geng, and J. Weng. "Automated federated learning-based adversarial attack and defence in industrial control systems." In: *IET Cyber-Systems and Robotics* 6.2 (May 2024). DOI: 10.1049/csy2.12117.

[68] C. Zhang, X. Yuan, Q. Zhang, G. Zhu, L. Cheng, and N. Zhang. "Privacy-preserving neural architecture search across federated IoT devices." In: *2021 IEEE 20th international conference on trust, security and privacy in computing and communications (TrustCom)*. 2021, pp. 1434–1438. DOI: 10.1109/TrustCom53373.2021.00203.

[69] C. Zhang, X. Yuan, Q. Zhang, G. Zhu, L. Cheng, and N. Zhang. "Toward tailored models on private AIoT devices: Federated direct neural architecture search." In: *IEEE Internet of Things Journal* 9.18 (2022), pp. 17309–17322. DOI: 10.1109/JIOT.2022.3154605.

[70] F. Zhang, J. Ge, C. Wong, S. Zhang, C. Li, and B. Luo. "Optimizing Federated Edge Learning on Non-IID Data via Neural Architecture Search." In: *2021 IEEE Global Communications Conference (GLOBECOM)*. 2021, pp. 1–6. DOI: 10.1109/GLOBECOM46510.2021.9685909.

[71] F. Zhang, M. Li, J. Ge, F. Tang, S. Zhang, J. Wu, and B. Luo. "Privacy-Preserving Federated Neural Architecture Search with Enhanced Robustness for Edge Computing." In: *IEEE Transactions on Mobile Computing* 24.3 (2025), pp. 2234–2252. DOI: 10.1109/TMC.2024.3490835.

[72] Z. Zhang, Z. Liu, Y. Zhao, C. Qiu, C. Zhang, and X. Wang. "ENASFL: A federated neural architecture search scheme for heterogeneous deep models in distributed edge computing systems." In: *IEEE Transactions on Network Science and Engineering* 11.3 (2024), pp. 2610–2620. DOI: 10.1109/TNSE.2023.3344850.

[73] Z. Zhao, C. Qiu, Y. Zhao, X. Wang, H. Yao, X. Li, and F. R. Yu. "F2NAS: Flexible federated neural architecture search in green edge computing." In: *ICC 2024 - IEEE international conference on communications*. 2024, pp. 1545–1550. DOI: 10.1109/ICC51166.2024.10623100.

[74] H. Zhu and Y. Jin. "Multi-Objective Evolutionary Federated Learning." In: *IEEE Transactions on Neural Networks and Learning Systems* 31.4 (2020), pp. 1310–1322. DOI: 10.1109/TNNLS.2019.2919699.

[75] H. Zhu and Y. Jin. *Real-time federated evolutionary neural architecture search*. 2020. arXiv: 2003.02793 [cs.LG].

[76] H. Zhu, H. Zhang, and Y. Jin. "From federated learning to federated neural architecture search: a survey." In: *Complex and Intelligent Systems* 7.2 (Apr. 2021), pp. 639–657. ISSN: 2198-6053. DOI: 10.1007/s40747-020-00247-z.